HERDR-MOBILE-1 App móvil de T3 Code sobre herdr-core

> ***How to read this PRD — precedence.*** *The intent sections (Requirements Analysis, Resolved Decisions, Out of Scope, and the contracts — API Contract, DB Schema Reference, Access Control, Validation & Error Handling, Definition of Done) are* ***binding****: the implementation MUST satisfy them. Technical Approach, Pattern to follow, and Conventions to follow are the* ***recommended path*** *— follow them to minimize guesswork, but if a step conflicts with the binding intent, the intent wins; deviate only with a noted reason.*

Overview

Fork de T3 Code (`pingdotgg/t3code` , MIT) cuya app móvil (`apps/mobile` , Expo 58 / React Native 0.88, iOS, iPadOS y Android) se conecta a **herdr-core** (`hc`), el runtime de terminales detrás de la app de escritorio Herdr, en lugar de a un servidor T3. El usuario trabaja siempre en sus servidores (`mylinux` , `gx10`) y quiere ver y operar desde el teléfono **los mismos espacios, pestañas, agentes y terminales** que en su escritorio, con la fluidez del chat de T3 móvil para Claude Code y Codex.

Requisito transversal: **el fork debe poder recibir las actualizaciones de T3 cada semana sin romper la integración.** T3 móvil recibió 874 commits en 30 días (`apps/mobile`), 341 en `packages/contracts` y 433 en `packages/client-runtime` (medido el 2026-11-05 sobre `337c5858c3`). Por eso la integración vive en código propio y aislado, y toca la menor cantidad posible de archivos de T3.

Tres piezas:

1. **`packages/herdr`** : cliente TypeScript del protocolo de `hc` y tipos del API del gateway.
2. **`apps/herdr-gateway`** : servicio que corre en cada servidor, habla con `hc connect` localmente y expone un API HTTP/WebSocket en la red de Tailscale. Incluye el proxy `herdr-tap` y el script de hooks de Claude Code, que alimentan el chat.
3. **`apps/mobile/src/features/herdr`** : la superficie herdr de la app, que reutiliza la terminal nativa, el layout adaptativo, el visor de archivos y la pantalla de chat de T3.

Requirements Analysis

Base de la app (v1):

- Al abrir la app se ve Herdr: agentes que necesitan atención arriba, y debajo cada máquina con sus carpetas (`space-folders`), espacios y pestañas. Las pantallas de T3 (Chats, PRs, Usage, proyectos) quedan ocultas, no borradas.
- El usuario agrega cada servidor una vez (URL del gateway en la red de Tailscale). La app se conecta a todos los gateways guardados a la vez. Si uno se cae, los demás siguen.
- Lista de agentes por prioridad, con los estados de herdr-core: `blocked` > `done` > `working` > `idle` (`error` cuenta como `blocked` : necesita atención y avisa). Un agente es una terminal cuyo `status` no es nulo.
- Terminal en vivo de cualquier panel: leer, escribir, teclas especiales, `^C` , ajuste al tamaño del teléfono y reconexión automática. Si una pestaña tiene varios paneles, se ven de uno en uno, deslizando.

- Abrir una pestaña nueva en un espacio existente, con un shell o con un agente (`claude` , `codex`).
- Archivos del espacio: navegar, previsualizar (texto, código, imágenes), descargar al teléfono y subir desde el teléfono.
- Aviso dentro de la app (con la app abierta) cuando un agente pasa a `blocked` o `done` .
- En iPad, columnas: barra lateral con la lista, contenido con el agente y un inspector de archivos, usando el layout adaptativo de T3.

Chat (después de la base):

- Vista de chat nativa para agentes de **Claude Code**, sobre la misma sesión que corre en la terminal de herdr: lo que se escribe en el teléfono aparece en el escritorio y al revés.
- Respuestas palabra por palabra, herramientas en vivo, botones para aprobar permisos y responder preguntas, enviar mensajes y detener el turno.
- Vista de chat para **Codex**, primero comprobando qué fuente de datos es viable (ver Resolved Decisions).
- Selector Chat / Terminal en cada agente. Los agentes que no son Claude Code ni Codex solo tienen terminal.

Notificaciones push (última etapa, historias 17 y 18, bloqueadas hasta tener cuentas): aviso con la app cerrada cuando un agente pasa a `blocked` o `done` , en iOS (APNs) y Android (FCM).

Mantenimiento del fork: un job semanal abre un PR con los cambios de T3 y corre las pruebas; un chequeo de CI rechaza cambios a archivos de T3 que no estén en la lista permitida.

Resolved Decisions

1. **Q: ¿Qué base?** → **Decision:** T3 Code directo (`pingdotgg/t3code`), no `benkraus/herdr-mobile` . Motivo: en `benkraus` solo ~8,000 de 60,500 líneas se ejecutan, y el chat de T3 es código muerto de hace 7,209 commits. Con T3 hay historial común y se puede hacer `git merge` .
2. **Q: ¿Arquitectura?** → **Decision:** superficie herdr aislada. Todo el código nuevo vive en `packages/herdr/` , `apps/herdr-gateway/` , `apps/mobile/src/features/herdr/` , `docs/herdr/` , `scripts/herdr/` y `.github/workflows/herdr-*.yml` . Los archivos de T3 solo se modifican si están en `docs/herdr/patch-surface.txt` . Nada de T3 se borra: lo que no se use se oculta. Ejemplo: para que el inicio sea Herdr se cambia el `screen` de la ruta `Home` en `apps/mobile/src/Stack.tsx` ; `WorkspaceRootScreen` sigue existiendo y compilando.
3. **Q: ¿Qué se ve al abrir?** → **Decision:** Herdr es la pantalla principal (ruta `Home`). Las pestañas de T3 no aparecen.
4. **Q: ¿Alcance de la v1?** → **Decision:** espacios, pestañas, terminal en vivo, agentes por prioridad, archivos (navegar, ver, descargar, subir) y avisos con la app abierta. Las notificaciones push van al final y bloqueadas (decisión 13). El chat va justo después de la base (decisión 8).
5. **Q: ¿Cómo se alimenta el chat de Claude Code con los mismos agentes del escritorio?** → **Decision:** dos fuentes oficiales, con el Claude Code real intacto en su terminal:
   - **Proxy** **`herdr-tap`** vía `ANTHROPIC_BASE_URL` : texto y herramientas palabra por palabra, correlacionados con el header `x-claude-code-session-id` .
   - **Hooks de Claude Code**: `SessionStart` , `UserPromptSubmit` , `PreToolUse` , `PostToolUse` , `PermissionRequest` , `Notification` , `Stop` , `SessionEnd` .
   - **Transcripción** (`~/.claude/projects/<proyecto>/<session-id>.jsonl`): solo para el historial al abrir el chat, porque su formato es interno y puede cambiar.
   - **Envío desde el teléfono**: `terminal.send` de herdr-core, sobre la misma terminal.
6. **Q: ¿A qué sesiones se aplica el proxy?** → **Decision:** a todo Claude Code que se lance desde zsh en el servidor, sin cambiar nada más del entorno. `~/.zshenv` carga `herdr-tap-env.sh` , que solo define una función de zsh `claude` ; esa función corre `command claude` con `ANTHROPIC_BASE_URL=http://127.0.0.1:7790` puesta solo

para ese proceso, y solo si en ese momento se cumplen tres condiciones: (a) `ANTHROPIC_BASE_URL` no está definida, (b) el archivo PID de `herdr-tap` existe y el proceso responde a `kill -0`, y (c) `/proc/<pid>/cmdline` de ese PID contiene `herdr-tap`. Si alguna falla, corre `command claude` sin tocar nada y Claude va directo a Anthropic. Ninguna otra variable ni programa cambia. Ejemplo: `claude-glm` y `flashgate`, que exportan su propia `ANTHROPIC_BASE_URL`, siguen su ruta y no tienen chat palabra por palabra.

7. **Q: ¿Codex? → Decision:** el chat de Codex entra al plan, con comprobación primero. Base garantizada: leer su registro de sesión (`~/.codex/sessions/**/rollout-*.jsonl`), compatible con `--no-daemon`, con respuestas por bloques. Palabra por palabra solo si se demuestra que una terminal de herdr con Codex y el gateway pueden compartir un `app-server` sin romper la regla de herdr de que Codex corre con `--no-daemon`. Si no, se queda la base y se documenta por qué.
8. **Q: ¿Cuándo va el chat? → Decision:** justo después de la base (olas 3 a 6 del breakdown). Las historias 11 y 12 comprueban primero, con Claude real, que el proxy y los hooks funcionan.
9. **Q: ¿Un gateway central o uno por servidor? → Decision:** uno por servidor. Cada servidor corre su gateway, su `herdr-tap` y sus hooks. La app guarda una lista de gateways y se conecta a cada uno. Ejemplo: si `gx10` se apaga, `mylinux` sigue visible y `gx10` aparece como desconectado.
10. **Q: ¿Tamaño de la terminal? → Decision:** se ajusta **siempre** al teléfono. La app manda su tamaño en celdas al abrir y al rotar. En herdr-core, el último visor activo define el tamaño, así que el escritorio se reacomoda mientras el teléfono la usa y lo recupera cuando el escritorio vuelve a enfocarla.
11. **Q: ¿Seguridad del gateway? → Decision:** solo Tailscale. El gateway escucha únicamente en la IP de Tailscale del servidor (`tailscale ip -4`), nunca en `0.0.0.0`. Además, en cada conexión HTTP y WebSocket consulta `tailscale whois --json <ip>:<puerto>` del cliente y acepta solo si `UserProfile.LoginName` coincide con el dueño del propio nodo. Si no coincide, responde `403`. Ejemplo verificado: la MacBook del usuario resuelve a `kuamatzin@gmail.com`, igual que `mylinux`. Un dispositivo de las socias de Protact sería rechazado aunque vea el puerto.
12. **Q: ¿Actualizaciones de T3? → Decision:** semanales y revisadas. Un workflow abre cada semana un PR de `pingdotgg/t3code@main` hacia el fork, corre el chequeo de archivos permitidos y las pruebas de `packages/herdr`, `apps/herdr-gateway` y `apps/mobile/src/features/herdr`. Si hay conflictos, el PR se abre como borrador listando los archivos en conflicto. Nunca se fusiona solo.
13. **Q: ¿Notificaciones push sin cuentas de Apple ni Firebase? → Decision:** historias 17 (gateway) y 18 (app), bloqueadas (🚧) hasta que el usuario tenga Apple Developer de pago y un proyecto de Firebase. Mientras tanto, la v1 avisa solo con la app abierta.
14. **(plan) Gateway sin Effect.** El gateway y `packages/herdr` usan TypeScript plano con Promises y APIs de Node, no Effect, para no acoplarse a la evolución de las librerías del servidor T3. `packages/herdr` no importa nada de `@t3tools/*`.
15. **(plan) El proxy es un proceso aparte.** `herdr-tap` corre como su propio servicio systemd y reenvía a `https://api.anthropic.com` sin depender del gateway. Si el gateway se reinicia o cae, los agentes no se cortan. `herdr-tap` nunca guarda credenciales ni cuerpos de petición o respuesta; solo emite eventos por un socket Unix local, sin esperar respuesta.
16. **(plan) Archivos sin pasar por `hc`.** El gateway corre en el servidor, así que lee y escribe el disco con `node:fs`, limitado a rutas dentro de `$HOME` después de resolver enlaces (`realpath`).
17. **(plan) Aprobaciones sin bloquear al escritorio.** El hook solo espera la decisión del teléfono si hay al menos una app con el chat de ese agente abierto en ese momento. Si no, responde de inmediato sin decidir y Claude sigue su flujo normal en la terminal. La espera máxima es 120 s; al vencer, también sigue el flujo normal. Una espera también se libera sin decidir al llegar, para la misma sesión, cualquiera de estos eventos: `PostToolUse`, `PostToolUseFailure`, `PermissionDenied`, `Stop`, `UserPromptSubmit` o `SessionEnd` (nunca `Notification`). Respaldo aprobado: si la comprobación con Claude real muestra que responder en la terminal no termina la espera del hook, la espera máxima baja a 30 s y se anota en `docs/herdr/hooks.md` y en el tracker.

18. **(plan) Planes dentro del fork.** El `AGENTS.md` de T3 pide no subir planes. En este fork, `docs/tasks/` es una excepción documentada en `docs/herdr/AGENTS.md`, porque son carpetas nuevas y no chocan con T3.
19. **(plan) Sugerencias de permiso solo informativas.** Las `permission_suggestions` de Claude Code se muestran en la tarjeta solo como información; desde el teléfono solo se responde `allow` o `deny`, porque el contrato de `/approve` no tiene cómo elegir una sugerencia.
20. **(plan) Aprobaciones solo con el chat abierto.** Una aprobación o pregunta llega al teléfono solo mientras el chat de ese agente está abierto en el teléfono. Una petición que se levantó antes de abrirlo, o mientras se ve en modo Terminal, se queda en la terminal. Es el comportamiento buscado y se prueba.

Out of Scope

- Borrar archivos desde la app.
- Crear, renombrar o eliminar espacios; renombrar o cerrar pestañas. Solo se abren pestañas nuevas.
- Ver varios paneles de una pestaña al mismo tiempo.
- Live Activities, widgets y la pantalla de uso de T3.
- Chat para agentes que no sean Claude Code o Codex. Se ven como terminal.
- Escribir en la terminal en modo espejo sin ajustar el tamaño (decisión 10).
- Usar `ssh` desde el teléfono; el teléfono solo habla con el gateway.
- Conectar la app a servidores T3 normales. El código de T3 sigue ahí, pero no se prueba ni se mantiene.
- Mostrar el árbol de máquinas de `hc machine` (máquinas que un core alcanza por SSH). Cada gateway representa a su propio servidor.
- Publicar en App Store o Google Play.
- Mantener o actualizar el `herdr` clásico.

User Flow

1. **Primera vez:** en cada servidor, el usuario corre `apps/herdr-gateway/scripts/install.sh`, que instala y arranca `herdr-gateway` y `herdr-tap` como servicios systemd de usuario, agrega los hooks a `~/.claude/settings.json` y a `~/.zshenv` la línea que define la función `claude` (decisión 6). El script imprime la URL del gateway.
2. En la app, "Agregar servidor": pega la URL (`http://100.x.y.z:7788`). La app llama `GET /v1/health`, muestra el nombre del servidor y lo guarda.
3. **Inicio:** arriba, "Necesitan atención" (agentes `blocked` y `done`), luego "Trabajando". Debajo, cada servidor con sus carpetas y espacios; cada espacio despliega sus pestañas.
4. Tocar un agente o pestaña abre su panel. Si el agente es Claude Code o Codex, abre en Chat; si no, en Terminal. Un selector permite cambiar.
5. **Terminal:** la terminal se ajusta al teléfono. Barra de teclas (`esc`, `tab`, `ctrl`, flechas, `^C`). Si la pestaña tiene varios paneles, se desliza entre ellos.
6. **Chat:** historial, la respuesta en curso palabra por palabra, herramientas plegadas. Si Claude pide permiso mientras el chat está abierto, aparece una tarjeta con "Permitir" y "Denegar" (las sugerencias de Claude solo como información, decisión 19); al tocar uno, la terminal sigue. Escribir un mensaje lo manda a la misma terminal.
7. **Archivos:** desde el espacio o el inspector en iPad: navegar, ver, descargar (hoja de compartir del sistema) y subir (selector de documentos).
8. **Pestaña nueva:** en un espacio, "+" → Shell, Claude o Codex. La pestaña aparece también en el escritorio.

9. Si un agente pasa a `blocked` o `done` con la app abierta, aparece un aviso que lleva a ese agente.

Affected Components

Frontend

- Files to modify (T3, solo estos): `apps/mobile/src/Stack.tsx` (ruta `Home` y rutas herdr), un archivo del layout para la barra lateral (`apps/mobile/src/features/layout/AdaptiveWorkspaceLayout.tsx` o `apps/mobile/src/features/workspace/WorkspaceThreadSidebar.tsx` , el que requiera el cambio más pequeño), `apps/mobile/app.config.ts` , `apps/mobile/app.config.test.ts` , `apps/mobile/package.json` .
- Pattern to follow:
  - Terminal: `apps/mobile/src/features/terminal/ThreadTerminalRouteScreen.tsx` y el componente `TerminalSurface` de `apps/mobile/src/features/terminal/NativeTerminalSurface.tsx` (props `terminalKey` , `buffer` , `output` , `grid` , `isRunning` , `onInput` , `onResize`).
  - Archivos: `apps/mobile/src/features/files/` (`FileTreeBrowser.tsx` , `SourceFileSurface.tsx` , `WorkspaceFileImagePreview.tsx`).
  - Chat: `apps/mobile/src/features/threads/ThreadFeed.tsx` (`ThreadFeedProps.feed: ReadonlyArray<ThreadFeedEntry>` , tipo en `apps/mobile/src/lib/threadActivity.ts:192`) y `ThreadComposer.tsx` .
  - Layout: `AdaptiveWorkspaceLayout` (envuelto por `RootStackLayout` en `Stack.tsx`) y su contexto `useAdaptiveWorkspaceLayout` .
  - Barra lateral: `apps/mobile/src/features/workspace/WorkspaceThreadSidebar.tsx` .
- Conventions to follow: estado con átomos de `@effect/atom-react` como el resto de `apps/mobile/src/state` , o hooks de React con estado local donde baste; nombres de archivo como los vecinos (`PascalCase.tsx` para componentes, `kebab-case.ts` o `camelCase.ts` para lógica); estilos con clases Uniwind (`className`) y `useThemeColor` ; símbolos con `AppSymbol` ; pruebas `*.test.ts` junto al archivo.
- New components needed: `HerdrHomeRoute` , `HerdrHostSheet` , `HerdrAgentList` , `HerdrSpaceTree` , `HerdrSidebar` , `HerdrAgentRoute` (Chat/Terminal), `HerdrTerminalPane` , `HerdrFilesRoute` , `HerdrChatScreen` , `HerdrChatComposer` , `HerdrApprovalCard` .
- UI primitives to use: `AppText` , `AppSymbol` , `AppBar` , componentes de `apps/mobile/src/components/ui` , `TerminalSurface` , `ThreadFeed` , módulos nativos `t3-terminal` , `t3-composer-editor` .
- State changes: nuevo `apps/mobile/src/features/herdr/state/` con registro de gateways (`expo-secure-store`), conexión por gateway, snapshot y eventos.
- Services: cliente HTTP/WebSocket del gateway en `apps/mobile/src/features/herdr/state/gatewayClient.ts` , tipado con `packages/herdr/src/gateway-api.ts` .

Backend

- API endpoints: ver `## API Contract` .
- Pattern to follow: `apps/server/src/terminal/OutputProtocol.ts` y `withTerminalOutputWindow` como referencia de control de flujo; `apps/server/src/relay/` y `infra/relay/src/agentActivity/ApnsClient.ts` como referencia de APNs. El protocolo de `hc` está en `https://github.com/herdrdev/herdr-core/blob/main/schema/README.md` y en `hc schema` .
- Conventions to follow: TypeScript plano (decisión 14), solo los tipos que se borran (sin `enum` ni `namespace`), errores como `{ error: { kind, message } }` con códigos HTTP explícitos.

- Handlers / functions: `apps/herdr-gateway/src/` (ver Technical Approach).
- Database changes: ninguna. El estado del gateway vive en memoria, salvo los dispositivos push en `~/.config/herdr-gateway/devices.json`.
- Services: `herdr-gateway`, `herdr-tap` (systemd de usuario), script `herdr-hook`.
- Estado en disco del gateway: `~/.config/herdr-gateway/devices.json` y `push.env` (historia 17), y el mapa de sesiones en `$XDG_RUNTIME_DIR/herdr-gateway/sessions.json` (historia 12).

Technical Approach

Prerrequisitos para cualquier historia: `vp` (vite-plus) instalado según `docs/operations/development.md`, `vp i` sin errores en la raíz. Para validar se usa `vp test run <archivos>` y `vp run typecheck` del paquete afectado; nunca chequeos de todo el repo (regla del `AGENTS.md` de T3). Los builds iOS solo se pueden hacer en la Mac del usuario; Android se compila en `mylinux` (`~/Android/Sdk`).

Story 1: Protección de actualizaciones de T3

- **Files:** `docs/herdr/AGENTS.md`, `docs/herdr/patch-surface.txt`, `scripts/herdr/check-patch-surface.ts`, `.github/workflows/herdr-guard.yml`, `.github/workflows/herdr-upstream-sync.yml`, `CLAUDE.md` (una línea).
- **What to do:**
  1. `docs/herdr/AGENTS.md`: reglas del fork: directorios propios, lista permitida, no borrar código de T3, excepción de `docs/tasks/` (decisión 18), y cómo correr las pruebas propias.
  2. `docs/herdr/patch-surface.txt`: una ruta por línea, con comentario `#` de por qué. Inicial: `CLAUDE.md`, `apps/mobile/src/Stack.tsx`, `apps/mobile/app.config.ts`, `apps/mobile/app.config.test.ts`, `apps/mobile/package.json`, `pnpm-lock.yaml`.
  3. `scripts/herdr/check-patch-surface.ts`: calcula `git merge-base HEAD upstream/main`, lista los archivos modificados o borrados desde ahí que **existen en upstream** (`git cat-file -e upstream/main:<ruta>`), y falla si alguno no está en la lista permitida. Los archivos nuevos que no existen en upstream no cuentan. Sale con código 1 e imprime los archivos ofensores. Solo módulos de Node y sintaxis de TypeScript que Node ejecuta sin banderas (sin `enum`, `namespace` ni parámetros de propiedad); se corre con el Node que instala `vp` (versión fijada en `package.json` `engines`).
  4. `.github/workflows/herdr-guard.yml`: en `push` y `pull_request`, agrega el remoto `upstream` (`https://github.com/pingdotgg/t3code.git`), hace `fetch` y corre el script con el Node de `vp`.
  5. `.github/workflows/herdr-upstream-sync.yml`: `schedule` semanal (lunes 14:00 UTC) y `workflow_dispatch`. Crea la rama `upstream-sync/<AAAA-MM-DD>` y corre `git merge upstream/main` dentro de un `if` explícito (para que un conflicto no aborte el script). Si hay conflictos, hace commit con los marcadores y abre el PR como borrador con la lista de archivos en conflicto. Si no, **siempre** corre `vp i`, el chequeo de lista permitida, `vp run typecheck` de `packages/herdr`, `apps/herdr-gateway` y `apps/mobile`, y `vp test run` de los directorios herdr; abre el PR y lo marca como borrador si cualquiera de esos pasos falla, con el resultado de cada uno en el cuerpo. Usa `gh pr create`. Un job aparte, manual (`workflow_dispatch`) y no requerido, compila Android.
  6. `CLAUDE.md`: agregar al final la línea `@docs/herdr/AGENTS.md`.
  7. Documentar en `docs/herdr/AGENTS.md` que los workflows de T3 se deshabilitan en el fork con `gh workflow disable <nombre>` (acción del usuario, no edición de archivos).
- **Patterns to follow:** scripts existentes en `scripts/` (Node con TypeScript, `node scripts/<x>.ts`), workflows existentes en `.github/workflows/ci.yml` y la acción `./.github/actions/setup-ci-deps`.

Story 2: Identidad propia de la app

- **Files:** `apps/mobile/app.config.ts`, `apps/mobile/app.config.test.ts`, `docs/herdr/mobile-build.md`.
- **What to do:**
  1. En `VARIANT_CONFIG`: nombre `Herdr` (variantes `Herdr Dev`, `Herdr Preview`), bundle/package `com.kuamatzin.herdrmobile` con sufijos `.dev` y `.preview`, esquemas `herdr`, `herdr-dev`, `herdr-preview`.
  2. Desactivar por defecto lo que apunta a T3: `updates.enabled` en `false` salvo `HERDR_MOBILE_UPDATES_ENABLED=1`; quitar o condicionar `associatedDomains`, el intent filter de `HOSTED_APP_HOST`, `owner`, `extra.eas.projectId` y `updates.url` de T3; `appleTeamId` desde `HERDR_APPLE_TEAM_ID` (vacío por defecto).
  3. Ajustar `app.config.test.ts` a los valores nuevos.
  4. `docs/herdr/mobile-build.md`: cómo compilar Android en `mylinux` y iOS en la Mac con Personal Team (`T3CODE_IOS_PERSONAL_TEAM=1`), sin `.env.example` de T3.
- **Patterns to follow:** la estructura existente de `VARIANT_CONFIG` y `repoEnv` en `app.config.ts`. Mantener el diff mínimo: cambiar valores, no reestructurar.

Story 3: Cliente de `hc`

- **Files:** `packages/herdr/package.json`, `packages/herdr/tsconfig.json`, `packages/herdr/src/hc/frames.ts`, `packages/herdr/src/hc/connection.ts`, `packages/herdr/src/hc/types.ts`, `packages/herdr/fixtures/hc-schema.json`.
- **What to do:**
  1. Paquete `@herdr/core-client` (privado), sin dependencias de `@t3tools/*`, exportando `./hc` y `./gateway-api`.
  2. `frames.ts`: codificador y decodificador incremental de tramas `[channel u32 LE][kind u8][len u32 LE][payload]`, kinds `DATA=0, CREDIT=1, CLOSE=2, RESIZE=3, FOCUS=4, FACTS=5, AUTOMATIC_INPUT=6`, payload máximo 16 MiB.
  3. `connection.ts`: `HcConnection` sobre un transporte de bytes inyectado (`{ write(bytes), onData(cb), onClose(cb) }`). Hace `hello` en modo plano con `{ framed: true, generation: 1 }`, cambia a modo con tramas (JSON-RPC por el canal 0), y ofrece `call(method, params)`, `subscribe()` (eventos `event`, `heartbeat`, `missed`, por el canal que devuelve `events.subscribe`) y `attach(terminal, { cols, rows, history })`, que devuelve `{ onData, write, resize, consumed, close, onClose }`. Control de flujo según la especificación: el core da a cada stream 256 KiB de crédito; el visor manda `CREDIT` por los bytes que ya consumió; el cliente nunca manda un `CREDIT` inicial. Si se cae el transporte, cada stream abierto recibe `onClose` con razón `connection_lost`.
  4. `types.ts`: tipos de `Terminal` (incluye `foreground: {pid, name, argv, cwd}`, donde `pid` es el líder del grupo de procesos en primer plano), `Status`, `HelloResult`, `AttachResult`, `SubscribeResult` y `ReadResult` tomados de `fixtures/hc-schema.json`.
  5. Fuentes de verdad: la especificación del protocolo copiada en `docs/tasks/herdr-core-mobile/hc-protocol.md` y el esquema `docs/tasks/herdr-core-mobile/hc-schema-0.1.18.json` (copia de `hc schema` de herdr-core 0.1.18, generación 1), que se copia a `packages/herdr/fixtures/hc-schema.json`. No se infiere el formato de la red.
  6. Pruebas: tramas partidas en cualquier punto, varias tramas en un solo chunk, `hello` y cambio de modo, llamadas en paralelo, créditos y `CLOSE`. Una prueba de contrato verifica que cada método y parámetro requerido que usa el cliente o el gateway existe en el fixture (incluido `terminal.get`). Si `HERDR_LIVE_HC=1`, corre `hc schema` y compara con el fixture.

- **Patterns to follow:** `docs/tasks/herdr-core-mobile/hc-protocol.md`. Referencia de diseño (Apache-2.0, solo lectura): `zzulanas/herdr-ios` `Packages/HerdrKit/Sources/HerdrKit/HerdrClient.swift` y `Frame.swift`. **No copiar** código de `juanibiapina/herdr-mobile` (sin licencia).

Story 4: Gateway base

- **Files:** `apps/herdr-gateway/package.json`, `apps/herdr-gateway/src/main.ts`, `apps/herdr-gateway/src/http.ts`, `apps/herdr-gateway/src/auth.ts`, `apps/herdr-gateway/src/snapshot.ts`, `apps/herdr-gateway/src/routes/terminals.ts` (stub), `apps/herdr-gateway/src/routes/files.ts` (stub), `apps/herdr-gateway/src/routes/chat.ts` (stub), `apps/herdr-gateway/src/routes/devices.ts` (stub), `apps/herdr-gateway/src/push/notifier.ts` (stub), `apps/herdr-gateway/scripts/install.sh`, `packages/herdr/src/gateway-api.ts`.
- **What to do:**
  1. `main.ts`: lanza `hc connect` como proceso hijo y conecta su stdin/stdout a `HcConnection`; si el hijo muere, reintenta con espera creciente (1, 2, 4… hasta 30 s). Exige `hello.generation === 1`; si no, no usa la conexión y `/v1/health` responde `503 {"error":{"kind":"core_incompatible"}}`. Lee `HERDR_GATEWAY_PORT` (por defecto `7788`) y escucha solo en la IP de `tailscale ip -4`. Monta `GET /v1/health`, `GET /v1/snapshot` y `WS /v1/events`, y registra los routers de `routes/terminals.ts`, `routes/files.ts`, `routes/chat.ts` y `routes/devices.ts` (stubs que responden `501` hasta sus historias) y arranca el notificador de `push/notifier.ts` (stub que no hace nada hasta la historia 17). Así las historias 5, 6, 13 y 17 solo editan sus propios archivos.
  2. `http.ts`: router con `get`, `post`, `delete` y `ws` (handlers asíncronos), autorización antes de cualquier ruta, `json()` que lanza un error que el router convierte en `400`, y el contexto común.
  3. `auth.ts`: decisión 11. Cachea por IP del cliente durante 60 s. `tailscale whois --json` acepta `<ip>:<puerto>` o la IP sola (verificado). El dueño del nodo sale de `tailscale status --self --json` → `User[String(Self.UserID)].LoginName`.
  4. `snapshot.ts`: arma el modelo con `doc.get {name:"herdr-app"}`, `terminal.list` y `events.subscribe`. Espacios con nombre vacío se nombran como Herdr: raíz del repo git del cwd de su primera terminal, si no su carpeta, y `~` en el home. Carpetas desde `~/.config/herdr-space-folders.json` si existe: asignaciones explícitas en `of`, y si no, la carpeta cuya regla `match` alcanza más profundo en el cwd del espacio (portar `rule()` y `reach()` del mod `space-folders`, MIT, con crédito). Ante `missed`, rehace el snapshot. El lector del documento `herdr-app` valida los campos que usa; si la forma no es la esperada, `/v1/snapshot` responde `503 {"error":{"kind":"layout_unsupported"}}` en lugar de adivinar.
  5. `gateway-api.ts`: tipos del contrato de `## API Contract` (sin chat), incluido el registro de dispositivos.
  6. `scripts/install.sh`: compila el gateway a un solo archivo JS (el empaquetador que use `apps/server/package.json`, objetivo Node 22), lo copia a `~/.local/share/herdr-gateway/`, escribe `~/.config/systemd/user/herdr-gateway.service` con la ruta absoluta de `node` y `EnvironmentFile=-%h/.config/herdr-gateway/push.env` (opcional, para la historia 17), lo habilita y lo arranca, corre `hc schema` y avisa si difiere del fixture, e imprime la URL. Es idempotente.
  7. Pruebas: snapshot desde un `HcConnection` falso, nombres de espacio, carpetas, documento con forma inesperada, auth aceptada y rechazada, eventos `missed`.
- **Patterns to follow:** ninguno de T3 (decisión 14). Usar `node:http` y la librería `ws`.

Story 5: Gateway, terminal

- **Files:** `apps/herdr-gateway/src/routes/terminals.ts`, `apps/herdr-gateway/src/terminalStream.ts`.
- **What to do:** `WS /v1/terminals/:address/stream?cols=&rows=&history=`: `terminal.attach` con ese tamaño e `history` (por defecto 2000). Decodifica UTF-8 de forma incremental (`StringDecoder`) y manda `{type:"output",data}`. Recibe `{type:"input",data}` y `{type:"resize",cols,rows}`. Al cerrar el WebSocket

manda `CLOSE`; al llegar `CLOSE` del core manda `{type:"exit",reason,exitCode}`. Devuelve créditos a medida que entrega al socket, con un tope de 1 MiB sin entregar y una cola de créditos pendientes que se revisa hasta liberarse. Agrega `POST /v1/terminals/:address/send`, `/keys` e `/interrupt`, y `POST /v1/tabs`. Una pestaña nueva **debe aparecer en el escritorio**: según la documentación de herdr-app, una terminal con las etiquetas `herdr.space=<id>` y `herdr.tab=<id>` aparece en Herdr aunque la arranque otro (`hc run --keep --label herdr.space=<id> --label herdr.tab=<id> -- claude`) y un id que Herdr no conoce se vuelve una pestaña nueva; el id nuevo se genera al estilo de Herdr (`tb` + 32 hex). Pruebas con conexión falsa: UTF-8 partido, resize, cierre de ambos lados y créditos.

- **Patterns to follow:** `apps/server/src/terminal/OutputProtocol.ts` (ventanas de salida) como referencia.

Story 6: Gateway, archivos

- **Files:** `apps/herdr-gateway/src/routes/files.ts`, `apps/herdr-gateway/src/files.ts`.
- **What to do:** `GET /v1/files`, `/v1/files/content`, `/v1/files/download`, `POST /v1/files/upload` (ver API Contract). Todas las rutas se resuelven con `realpath` y deben quedar dentro de `$HOME` (decisión 16); los archivos se abren con `O_NOFOLLOW` en el último componente donde Node lo permite y el descriptor abierto se revalida (`realpath` de `/proc/self/fd/<fd>`). Al subir, si el nombre existe, se guarda como `nombre (1).ext`, luego `(2)`, etc.; se escribe a un `.part` oculto y se renombra al final. Pruebas: salida de `$HOME` por `..` y por enlace simbólico, conflicto de nombre, subida cortada sin dejar `.part`.
- **Patterns to follow:** la lógica de nombres libres del mod `files-download` del usuario (`stem (n).ext`).

Story 7: App, conexión e inicio

- **Files:** `apps/mobile/src/features/herdr/state/hosts.ts`, `apps/mobile/src/features/herdr/state/gatewayClient.ts`, `apps/mobile/src/features/herdr/state/useHerdrSnapshot.ts`, `apps/mobile/src/features/herdr/HerdrHomeRoute.tsx`, `apps/mobile/src/features/herdr/HerdrHostSheet.tsx`, `apps/mobile/src/features/herdr/herdrRoutes.tsx`, `apps/mobile/src/Stack.tsx`, `apps/mobile/package.json`.
- **What to do:**
  1. `hosts.ts`: lista de gateways en `expo-secure-store` (`herdr.hosts.v1`, JSON `[{id,url,name}]`), con las escrituras serializadas en una sola cola de promesas. Solo se aceptan URLs `scheme://host:puerto` (una URL con ruta se rechaza).
  2. `gatewayClient.ts`: `fetch` y `WebSocket` hacia un gateway, reconexión con espera creciente, estado `connecting | connected | offline`.
  3. `useHerdrSnapshot`: snapshot más eventos por gateway, combinados en un modelo por servidor.
  4. `HerdrHomeRoute`: por ahora lista simple por servidor (espacios y pestañas) y botón "Agregar servidor"; la historia 8 la completa.
  5. `HerdrHostSheet`: pegar URL, validar con `GET /v1/health`, guardar.
  6. `herdrRoutes.tsx`: rutas reservadas `HerdrAgent` (`herdr/:hostId/:address`) y `HerdrFiles` (`herdr/:hostId/files`), con pantallas temporales que las historias 9 y 10 reemplazan editando solo sus propios archivos.
  7. `Stack.tsx`: la ruta `Home` usa `HerdrHomeRoute` y se agregan las rutas de `herdrRoutes.tsx` con un solo spread. Es el único cambio a ese archivo en todo el plan, salvo lo que la historia 8 justifique.
  8. `package.json` de móvil: dependencia `@herdr/core-client: workspace:*`.
- **Patterns to follow:** `apps/mobile/src/features/connection/ConnectionsNewRouteScreen.tsx` para la hoja de agregar; `createNativeStackScreen` como en `Stack.tsx`.

Story 8: App, lista de agentes e iPad

- **Files:** `apps/mobile/src/features/herdr/HerdrHomeRoute.tsx`, `apps/mobile/src/features/herdr/HerdrAgentList.tsx`, `apps/mobile/src/features/herdr/HerdrSpaceTree.tsx`, `apps/mobile/src/features/herdr/HerdrSidebar.tsx`, `apps/mobile/src/features/herdr/useAgentAlerts.ts`, y el archivo del layout de la barra lateral (ver Affected Components).
- **What to do:** lista "Necesitan atención" (`blocked`, `done`) y "Trabajando", ordenada por prioridad y luego por cambio más reciente (`error` va con `blocked`); árbol servidor → carpeta → espacio → pestaña, con carpetas plegables; en iPad, `HerdrSidebar` reemplaza la barra lateral de T3 con el cambio más pequeño posible en el archivo del layout, que se agrega a `docs/herdr/patch-surface.txt` con su motivo. `useAgentAlerts` : con la app abierta, aviso cuando un agente pasa a `blocked`, `error` o `done`, que navega a `HerdrAgent` ; por servidor, el primer snapshot tras cada (re)conexión solo fija la línea base.
- **Patterns to follow:** `apps/mobile/src/features/workspace/WorkspaceThreadSidebar.tsx` y `ThreadNavigationSidebar`.

Story 9: App, terminal

- **Files:** `apps/mobile/src/features/herdr/HerdrAgentRoute.tsx`, `apps/mobile/src/features/herdr/HerdrTerminalPane.tsx`, `apps/mobile/src/features/herdr/state/useTerminalStream.ts`, `apps/mobile/src/features/herdr/HerdrNewTabSheet.tsx`.
- **What to do:** `HerdrAgentRoute` muestra los paneles de la pestaña con desplazamiento horizontal (uno a la vez). `HerdrTerminalPane` usa `TerminalSurface` con `terminalKey` = dirección; `useTerminalStream` adapta el WebSocket de la historia 5 al estado de salida que espera `TerminalSurface` (ver `packages/client-runtime/src/state/terminal.ts`). El stream se abre con un tamaño estimado desde el `onLayout` de la vista (la misma fórmula que `estimateGridSize` en `NativeTerminalSurface.tsx`) y se corrige con `resize` en cada `onResize`. Barra de teclas reutilizando la de `ThreadTerminalRouteScreen.tsx`. `HerdrNewTabSheet` : Shell, Claude (`["claude"]`) o Codex (`["codex","--no-daemon"]` , opción presente en codex 0.166) → `POST /v1/tabs`.
- **Patterns to follow:** `apps/mobile/src/features/terminal/ThreadTerminalRouteScreen.tsx`.

Story 10: App, archivos

- **Files:** `apps/mobile/src/features/herdr/HerdrFilesRoute.tsx`, `apps/mobile/src/features/herdr/state/useHerdrFiles.ts`, `apps/mobile/src/features/herdr/HerdrFilePreview.tsx`.
- **What to do:** navegar desde el cwd del espacio; previsualizar texto y código con el visor de T3 e imágenes con `WorkspaceFileImagePreview` (SVG se previsualiza como texto, no como imagen); descargar con `expo-file-system` y abrir la hoja de compartir con `expo-sharing` ; subir con `expo-document-picker` y `uploadAsync` de `expo-file-system/legacy` (`httpMethod: "POST"`, `uploadType: FileSystemUploadType.BINARY_CONTENT`). En iPad se muestra en el inspector.
- **Patterns to follow:** `apps/mobile/src/features/files/FileTreeBrowser.tsx`, `SourceFileSurface.tsx`, `WorkspaceFileImagePreview.tsx`.

Story 11: Proxy `herdr-tap`

- **Files:** `apps/herdr-gateway/src/tap/main.ts`, `apps/herdr-gateway/src/tap/sse.ts`, `apps/herdr-gateway/scripts/herdr-tap-env.sh`, `apps/herdr-gateway/scripts/install.sh` (agregar el servicio), `docs/herdr/tap.md`.

- **What to do:**
  1. `tap/main.ts`: escucha en `127.0.0.1:7790` y reenvía todo a `https://api.anthropic.com`. Conserva todos los headers de extremo a extremo (en especial `anthropic-version`, `anthropic-beta` y `authorization`), los códigos y los cuerpos de error, los headers `anthropic-ratelimit-unified-*`, `retry-after` y `x-should-retry`; quita solo los de salto. Las respuestas SSE se pasan evento por evento, sin buffer y sin quitar `ping`, respetando la contrapresión y propagando los cortes en ambos sentidos.
  2. En paralelo, sin retrasar el reenvío, `sse.ts` interpreta los eventos (`message_start`, `content_block_start`, `content_block_delta` con `text_delta`, `input_json_delta` y `thinking_delta`, `content_block_stop`, `message_stop`). Emite eventos JSON por el socket Unix `$XDG_RUNTIME_DIR/herdr-gateway/tap.sock` con `{sessionId, agentId, requestId, event, data}`, donde `sessionId` sale de `x-claude-code-session-id`. Si el socket no existe o falla, descarta en silencio.
  3. Nunca escribe a disco ni registra headers o cuerpos. Escribe su PID en `$XDG_RUNTIME_DIR/herdr-tap/pid`.
  4. `herdr-tap-env.sh`: define la función de zsh `claude` de la decisión 6 y nada más. `install.sh` agrega a `~/.zshenv` una línea idempotente que lo carga, y un servicio systemd `herdr-tap.service`.
  5. **Comprobación con Claude real** (criterio de aceptación): con el tap activo, una sesión interactiva de Claude Code en una terminal de herdr responde normal, y el socket recibe deltas de texto con el `sessionId` de esa sesión. Documentar el resultado en `docs/herdr/tap.md`.
- **Patterns to follow:** la especificación de gateways de Claude Code (`https://code.claude.com/docs/en/llm-gateway-protocol.md`).

Story 12: Hooks de Claude Code

- **Files:** `apps/herdr-gateway/scripts/herdr-hook.sh`, `apps/herdr-gateway/scripts/install-hooks.ts`, `apps/herdr-gateway/src/hooks.ts`, `apps/herdr-gateway/src/main.ts` (montar el receptor), `docs/herdr/hooks.md`.
- **What to do:**
  1. `herdr-hook.sh`: lee el JSON del hook por stdin y lo manda a `POST http://127.0.0.1:7789/v1/hooks` con los headers `x-herdr-terminal: $HC_TERMINAL_ID` (definida en el entorno de toda terminal de hc, verificado) y `x-herdr-event`. Para `PermissionRequest` y para `PreToolUse` con `tool_name == "AskUserQuestion"`, espera la respuesta e imprime lo que devuelva el gateway (decisión 17). En cualquier error, sale con código 0 sin imprimir nada.
  2. `install-hooks.ts`: agrega una entrada `command` por evento a `~/.claude/settings.json` **sin tocar las existentes**, marcada con un sufijo único para poder actualizarla o quitarla de forma idempotente sin tocar hooks ajenos, con respaldo `settings.json.bak-<fecha>` antes de escribir. Eventos: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, `PermissionDenied`, `Notification`, `Stop`, `SessionEnd`. El `timeout` de `PermissionRequest` y `PreToolUse` es de 130 s.
  3. `hooks.ts`: receptor en `127.0.0.1:7789` (nunca en Tailscale). Mantiene el mapa `sessionId → { terminal, transcriptPath, claudeVersion }`, lo guarda de forma atómica en `$XDG_RUNTIME_DIR/herdr-gateway/sessions.json` y lo recarga al arrancar. Para decisiones, guarda la petición pendiente con un `requestId` y resuelve con lo que mande la app (`allow` o `deny`; para `AskUserQuestion`, `allow` con `updatedInput` que incluye `questions` y `answers`). Libera las esperas según la decisión 17. La ruta de depuración para aprobar a mano existe solo con `HERDR_GATEWAY_DEBUG=1`, que nunca se pone en el servicio systemd.
  4. **Comprobación con Claude real** (criterio de aceptación): documentar en `docs/herdr/hooks.md` si el diálogo de permiso sigue visible en la terminal mientras el hook espera, y qué pasa si el usuario responde en la terminal primero. Si responder en la terminal no termina la espera del hook, aplicar el respaldo de la decisión 17 (espera máxima de 30 s).

- **Patterns to follow:** el hook que ya instala herdr en `~/.claude/hooks/herdr-agent-state.sh`, como ejemplo de cómo leer stdin. No modificar ese archivo.

Story 13: Gateway, sesión de chat

- **Files:** `apps/herdr-gateway/src/routes/chat.ts`, `apps/herdr-gateway/src/chat/session.ts`, `apps/herdr-gateway/src/chat/claudeTranscript.ts`, `apps/herdr-gateway/src/chat/tapSocket.ts`, `apps/herdr-gateway/src/main.ts` (dueño del socket del tap), `packages/herdr/src/gateway-api.ts` (tipos de chat).
- **What to do:** une por terminal los eventos del tap (texto en curso), de los hooks (prompts, herramientas, aprobaciones, fin de turno) y el historial de la transcripción (últimos 200 mensajes, lector tolerante: entradas desconocidas se ignoran; si falla, `historyUnavailable: true`). La transcripción se ubica solo con el `transcript_path` que dan los hooks (sin adivinar por cwd); sin él, `historyUnavailable: true`. Los eventos del tap de una sesión aún sin terminal se guardan por `sessionId` (máximo 30 s o 500 eventos) y se reproducen al llegar el mapeo; si no llega, se descartan. Entradas y salidas de herramientas se truncan a 4 KB. `main.ts` crea y cierra el socket del tap (directorio 0700, socket 0600). Expone `WS /v1/agents/:address/chat` y los `POST` de envío, interrupción y aprobación. Para enviar usa `terminal.send`; para detener, `terminal.interrupt`. Pruebas con secuencias grabadas de tap y hooks.
- **Patterns to follow:** el modelo de ítems de `## API Contract`.

Story 14: App, pantalla de chat

- **Files:** `apps/mobile/src/features/herdr/chat/HerdrChatScreen.tsx`, `apps/mobile/src/features/herdr/chat/herdrFeedEntries.ts`, `apps/mobile/src/features/herdr/chat/useHerdrChat.ts`, `apps/mobile/src/features/herdr/chat/HerdrChatFeed.tsx`.
- **What to do:** `HerdrChatFeed` (lista propia con los componentes de presentación de T3: markdown, bloques de código, ítems de herramienta) es el camino planeado. Antes, una comprobación acotada al inicio de la historia: `ThreadFeed` de T3 se usa solo si renderiza un fixture de entradas de usuario, asistente y herramienta (traducidas de `ChatItem` a `ThreadFeedEntry` por `herdrFeedEntries`, `apps/mobile/src/lib/threadActivity.ts`) con identificadores sintéticos (`environmentId = "herdr:<hostId>"`, `threadId = <address>`) usando solo los providers y átomos que ya monta toda la app, sin editar archivos de T3 fuera de la lista permitida. Cualquier provider o átomo requerido que falte = `HerdrChatFeed`. Se documenta cuál camino se tomó y por qué en las notas de la historia.
- **Patterns to follow:** `apps/mobile/src/features/threads/ThreadFeed.tsx` y `ThreadDetailScreen.tsx`.

Story 15: App, escribir y aprobar

- **Files:** `apps/mobile/src/features/herdr/chat/HerdrChatComposer.tsx`, `apps/mobile/src/features/herdr/chat/HerdrApprovalCard.tsx`, `apps/mobile/src/features/herdr/HerdrAgentRoute.tsx` (selector Chat/Terminal), `apps/mobile/src/features/herdr/chat/HerdrChatScreen.tsx`.
- **What to do:** editor con el módulo `t3-composer-editor` (enviar y detener); tarjeta de aprobación con `Permitir` / `Denegar` y las `permission_suggestions` como información (decisión 19), y preguntas de `AskUserQuestion` como opciones; selector Chat/Terminal (Chat por defecto para `app` `claude-code` o `codex`). Las aprobaciones solo llegan con el chat abierto (decisión 20).
- **Patterns to follow:** `apps/mobile/src/features/threads/ThreadComposer.tsx`.

Story 16: Chat de Codex

- **Files:** `apps/herdr-gateway/src/chat/codexRollout.ts`, `apps/herdr-gateway/src/chat/session.ts` (fuente Codex), `apps/herdr-gateway/src/routes/chat.ts` (aceptar `codex`), `docs/herdr/codex.md`.
- **What to do:**
  1. Comprobación: documentar en `docs/herdr/codex.md` si `codex --remote` contra un `app-server` compartido funciona dentro de una terminal de herdr sin violar `--no-daemon`. Es **viable** solo si Codex corre dentro de la terminal de herdr con sus comandos ejecutándose bajo hc (verificable con `hc ls`) y el gateway recibe los deltas de los ítems sin que el usuario cambie cómo lanza Codex. Si no, explicar por qué.
  2. Base: localizar el `rollout-*.jsonl` del Codex de una terminal por proceso: desde el `foreground.pid` de la terminal (`terminal.get`) bajar hasta el proceso `codex` y leer los enlaces de `/proc/<pid>/fd` buscando un `rollout-*.jsonl` abierto; solo si no se encuentra, recurrir al más reciente con `session_meta.cwd` igual al cwd de la terminal y creado después de la terminal, marcado con `sessionUncertain: true`. Traducir `response_item` (`message`, `custom_tool_call`, `custom_tool_call_output`, `reasoning`) y `event_msg` (`task_started`, `item_completed`) a `ChatItem`. El envío y la interrupción van por la terminal.
  3. Si la comprobación del paso 1 sale viable, agregar palabra por palabra con esa vía; si no, queda solo la base.
- **Patterns to follow:** `packages/effect-codex-app-server/` (solo como referencia del protocolo).

Story 17: Push en el gateway (🚧 bloqueada hasta tener cuentas)

- **Files:** `apps/herdr-gateway/src/push/notifier.ts`, `apps/herdr-gateway/src/push/devices.ts`, `apps/herdr-gateway/src/push/apns.ts`, `apps/herdr-gateway/src/push/fcm.ts`, `apps/herdr-gateway/src/routes/devices.ts`.
- **What to do:** registro de dispositivos (`POST /v1/devices` y `DELETE /v1/devices/:token`, guardados en `~/.config/herdr-gateway/devices.json`, modo 600, sin duplicar por `token`); aviso cuando un agente pasa a `blocked` o `done`, uno por (terminal, turno, estado), con título = espacio y cuerpo = mensaje del estado; APNs con llave `.p8` (`HERDR_APNS_KEY_PATH`, `HERDR_APNS_KEY_ID`, `HERDR_APPLE_TEAM_ID`, `HERDR_APNS_TOPIC`) y FCM HTTP v1 con cuenta de servicio (`HERDR_FCM_SERVICE_ACCOUNT`). Las credenciales viven en `~/.config/herdr-gateway/push.env` (modo 600), que el servicio carga con `EnvironmentFile=` (ya puesto por la historia 4); nunca se registran tokens ni llaves. Un dispositivo se borra con APNs `410` o FCM `UNREGISTERED`. Solo reemplaza los stubs de la historia 4: no toca `main.ts` ni `install.sh`.
- **Patterns to follow:** `infra/relay/src/agentActivity/ApnsClient.ts`.

Story 18: Push en la app (🚧 bloqueada hasta tener cuentas)

- **Files:** `apps/mobile/src/features/herdr/state/usePushRegistration.ts`, `apps/mobile/src/features/herdr/HerdrHomeRoute.tsx` (montarlo), `apps/mobile/app.config.ts` y `apps/mobile/app.config.test.ts` (capacidades de push).
- **What to do:** en la app, `expo-notifications` obtiene el token del dispositivo y lo registra en cada gateway (y lo borra con `DELETE /v1/devices/:token` al quitar un servidor); tocar el aviso abre `HerdrAgent`; con la app en primer plano no hay aviso del sistema. Capacidades de push en `app.config.ts` solo con `HERDR_PUSH_ENABLED=1`.
- **Patterns to follow:** `apps/mobile/src/features/agent-awareness/remoteRegistration.ts`.

API Contract

Base: `http://<ip-tailscale-del-servidor>:7788`. Todas las rutas exigen la autorización de la decisión 11 (`403 {"error":{"kind":"forbidden"}}` si falla). JSON en UTF-8.

`GET /v1/health` → `{ "gatewayVersion": "0.1.0", "hc": { "version": "0.1.18", "generation": 1 }, "machine": { "coreId": "2918…", "name": "mylinux" } }`

`GET /v1/snapshot` →

```
{
  "seq": 42,
  "machine": { "coreId": "2918…", "name": "mylinux" },
  "folders": [{ "id": "f1", "name": "Protact", "color": "ansi4", "folded": false, "spaceIds":
  "spaces": [{ "id": "sp…", "name": "odoo", "cwd": "/home/u/…/odoo", "folderId": "f1",
    "tabs": [{ "id": "tb…", "name": "1", "panes": [{ "paneId": "p…", "terminal": "t1" }] }] }]
  "terminals": [{ "address": "t1", "spaceId": "sp…", "tabId": "tb…", "cwd": "/home/u/…",
    "title": null, "turn": 18, "app": "claude-code",
    "status": { "state": "blocked", "kind": "permission", "msg": "Bash(npm test)" },
    "changedAtMs": 1793900000000 }]
}
```

`status` es `null` en terminales que no reportan estado. Los ids son los de herdr-core, sin prefijo de máquina.

`WS /v1/events` → primer mensaje `{ "type": "snapshot", ...igual que /v1/snapshot }`; luego `{ "type": "terminal", "terminal": {…} }`, `{ "type": "terminalRemoved", "address": "t3" }`, `{ "type": "layout", "spaces": [...], "folders": [...] }` y `{ "type": "heartbeat", "seq": 50 }` cada 15 s.

`WS /v1/terminals/:address/stream?cols=80&rows=24&history=2000`: servidor → `{ "type": "output", "data": "…" }`, `{ "type": "exit", "reason": "exited", "exitCode": 0 }`; cliente → `{ "type": "input", "data": "…" }`, `{ "type": "resize", "cols": 60, "rows": 30 }`.

`POST /v1/terminals/:address/send` `{ "text": "…" }` → `{ "turn": 19, "started": true }`. **`/keys`** `{ "keys": ["ctrl+c"] }`. **`/interrupt`** `{}`.

`POST /v1/tabs` `{ "spaceId": "sp…", "argv": ["claude"] }` (`argv` opcional; sin él, shell) → `{ "address": "t9", "tabId": "tb…" }`. El gateway usa `terminal.run` con `cwd` = cwd del espacio, `keep: true` y las etiquetas `herdr.space=<spaceId>` y `herdr.tab=<id nuevo>`.

`GET /v1/files?path=/abs/dir` → `{ "path": "/abs/dir", "entries": [{ "name": "src", "dir": true, "size": null, "mtimeMs": 0 }] }` (máximo 500 entradas; `"truncated": true` si hay más).
`GET /v1/files/content?path=` → `{ "text": "…", "truncated": false }` (máximo 1 MiB; `415 {"error":{"kind":"binary"}}` si tiene bytes nulos).
`GET /v1/files/download?path=` → bytes con `content-disposition: attachment`.
`POST /v1/files/upload?dir=/abs&name=a.png` (cuerpo: bytes) → `{ "path": "/abs/a (1).png" }`.

`WS /v1/agents/:address/chat` → primer mensaje `{ "type": "snapshot", "source": "claude" | "codex", "items": [ChatItem], "historyUnavailable": false, "streaming": true }`; luego `{ "type": "item", "item": ChatItem }` (alta o reemplazo por `id`), `{ "type": "delta", "id": "…", "text": "…" }` y `{ "type": "turnEnded" }`.

`ChatItem` es uno de (el gateway trunca `input` y `output` de herramientas a 4 KB):

- `{ "kind": "user", "id", "text", "atMs" }`
- `{ "kind": "assistant", "id", "text", "streaming": boolean }`
- `{ "kind": "tool", "id", "name", "input": object, "status": "running" | "done" | "error", "output": string | null }`
- `{ "kind": "approval", "id": requestId, "tool": string, "input": object, "suggestions": [...], "questions": [...] | null, "state": "pending" | "resolved" | "expired" }`

`POST /v1/agents/:address/chat/send` `{ "text": "…" }`. `/interrupt` `{}`. `/approve` `{ "requestId": "…", "decision": "allow" | "deny", "answers": { … } | null }` → `409 {"error":{"kind":"not_pending"}}` si ya se resolvió o venció.

Interno, solo `127.0.0.1:7789`: `POST /v1/hooks` (headers `x-herdr-terminal`, `x-herdr-event`; cuerpo: el JSON del hook) → `200` con la salida JSON del hook, o `204` para no decidir.

Interno, socket Unix `$XDG_RUNTIME_DIR/herdr-gateway/tap.sock`: datagramas `{ "sessionId", "agentId", "requestId", "event", "data" }`.

`POST /v1/devices` (historia 17) `{ "platform": "ios" | "android", "token": "…", "environment": "sandbox" | "production" }` → `204`.
`DELETE /v1/devices/:token` (historia 17; `token` codificado en la URL, máximo 4096 caracteres) → `204`; `404 {"error":{"kind":"not_found"}}` si no está registrado.

Compatibilidad (fail-closed): si `hc` no habla la generación 1, `GET /v1/health` responde `503 {"error":{"kind":"core_incompatible"}}`. Si el documento `herdr-app` no tiene la forma esperada, `GET /v1/snapshot` responde `503 {"error":{"kind":"layout_unsupported"}}`.

Design System / UI Compliance

- Components used: `AppText`, `AppSymbol`, `AppBar`, componentes de `apps/mobile/src/components/ui`, `TerminalSurface`, `ThreadFeed` (o sus piezas), editor `t3-composer-editor`.
- Design tokens: los de Uniwind y `useThemeColor` que usa T3 (`bg-screen`, `bg-primary`, etc.); colores de estado de agente: `working`, `blocked`, `done`, `idle` como en el resto de la app.
- Icons / assets: símbolos de `AppSymbol` (SF Symbols en iOS). Nada nuevo de marca de T3.

Access Control

- Un solo usuario: el dueño de la cuenta de Tailscale que también es dueño del servidor (decisión 11). Sin cuentas ni roles en la app.
- El gateway escucha solo en la IP de Tailscale; el receptor de hooks y `herdr-tap` solo en `127.0.0.1` y el socket Unix del usuario.
- Archivos limitados a `$HOME` del usuario del servidor después de `realpath` (decisión 16).
- `herdr-tap` nunca registra credenciales ni contenido.
- Modelo de confianza: los procesos locales del mismo usuario son de confianza (ya tienen acceso de shell). Las carreras TOCTOU con enlaces simbólicos hechas por procesos del mismo usuario quedan fuera del alcance; aun así, los archivos se abren con `O_NOFOLLOW` en el último componente donde Node lo permite. El directorio del socket del tap se crea con modo 0700 y el socket con 0600. La ruta de depuración del receptor de hooks que aprueba a mano existe solo con `HERDR_GATEWAY_DEBUG=1`, que nunca se pone en el servicio systemd.
- Credenciales de push en `~/.config/herdr-gateway/push.env` (modo 600), cargadas con `EnvironmentFile=`; tokens y llaves nunca se registran.
- Quien pasa la autorización puede escribir en terminales y lanzar procesos: equivale a acceso de shell. Por eso no hay modo de "solo lectura" en la v1.

Validation & Error Handling

- Gateway caído o fuera de la red: el servidor aparece como "Sin conexión" con la hora del último dato; los demás siguen. Reintentos con espera creciente hasta 30 s.
- `hc connect` muere: el gateway reintenta; mientras tanto `/v1/health` responde `503 {"error":{"kind":"core_unavailable"}}`.
- URL inválida al agregar servidor, o respuesta que no es de un gateway: mensaje "No es un gateway de Herdr" y no se guarda.
- Terminal que termina mientras se ve: mensaje "La terminal terminó (código N)" y botón para volver.
- `missed` en eventos: rehacer el snapshot sin perder la navegación.
- Rutas de archivos fuera de `$HOME`: `403 {"error":{"kind":"outside_home"}}`. Inexistente: `404 {"error":{"kind":"not_found"}}`. Archivo binario en `/content`: `415`. Más de 1 MiB: se trunca con `truncated: true`.
- Subida interrumpida: no queda el `.part`; la app muestra el error.
- Chat sin datos del tap (por ejemplo, sesiones con `claude-glm`): el chat funciona con hooks y transcripción, sin palabra por palabra, y lo indica con una línea "Sin streaming en vivo".
- Aprobación respondida en la terminal antes que en el teléfono: la tarjeta pasa a `resolved` y el botón ya no envía nada (`409`).
- Hook sin gateway o sin app conectada: no decide; Claude sigue su flujo normal (decisión 17).
- Aprobación levantada antes de abrir el chat, o con el agente en modo Terminal: se queda en la terminal (decisión 20).
- `hc` de otra generación o documento `herdr-app` con forma inesperada: `503 core_incompatible` / `503 layout_unsupported`, sin adivinar.
- Eventos del tap de una sesión aún sin terminal: se guardan hasta 30 s o 500 eventos y se reproducen al llegar el mapeo; si no, se descartan.

Suggested Story Breakdown

Sizing the breakdown — fewest stories that fit. Worker: `Sonnet 5` (presupuesto ~80–120K, ~6 archivos de producto, 3 áreas). 18 historias porque son cuatro sistemas independientes (gateway, app, proxy y hooks, push) con varias capacidades cada uno, y tres historias de riesgo propio que deben poder fallar sin bloquear el resto. Push se parte en gateway (17) y app (18) por tamaño y por área.

Waves — parallel implementation. Cada ola corre en paralelo; las historias de una misma ola no comparten archivos de producto. La historia 4 crea stubs de `routes/terminals.ts`, `routes/files.ts`, `routes/chat.ts`, `routes/devices.ts` y `push/notifier.ts` (y la línea `EnvironmentFile=` del servicio), y la 7 crea `herdrRoutes.tsx` con pantallas temporales (incluidas `HerdrNewTab` y `HerdrFilesInspector`), para que las olas 3 y 4 no compartan archivos de registro.

#	Title	Scope	Est. Effort	Depends On	Wave
1	Protección de actualizaciones de T3	Lista permitida, chequeo en CI, PR semanal de T3, `docs/herdr/AGENTS.md`, línea en `CLAUDE.md`	M	none	1
2	Identidad propia de la app	Nombre, bundle ID, sin OTA ni dominios de T3; compila en Android	S	none	1
3	Cliente de `hc`	`packages/herdr`: tramas, JSON-RPC, eventos, attach con créditos, fixture y prueba de contrato	M	none	1
4	Gateway base	Servicio por servidor, Tailscale + whois, health, snapshot, eventos, carpetas, stubs de rutas, instalación systemd; split: file cap	M	story-3	2
5	Gateway, terminal	Stream WS, input, resize, créditos, send/keys/interrupt, nueva pestaña; split: file cap	M	story-4	3
6	Gateway, archivos	Listar, leer, descargar, subir sin sobrescribir, límite a `$HOME`	S	story-4	3
7	App, conexión e inicio	Servidores guardados, cliente del gateway, Herdr como inicio, rutas reservadas; split: file cap	M	story-4	3
8	App, lista de agentes e iPad	Prioridad, árbol con carpetas, barra lateral en iPad, avisos con app abierta; split: file cap	M	story-7	4
9	App, terminal	`TerminalSurface` sobre el stream, teclas, ajuste al teléfono, deslizar paneles, nueva pestaña	M	story-5, story-7	4
10	App, archivos	Navegar, previsualizar, descargar y subir; inspector en iPad	M	story-6, story-7	4
11	Proxy `herdr-tap`	Pass-through a Anthropic, parser SSE, socket Unix, env en `.zshenv`, servicio; comprobación con Claude real; split: risk	M	story-4	3
12	Hooks de Claude Code	`herdr-hook`, instalación sin pisar hooks, receptor local, aprobaciones y preguntas; comprobación con Claude real; split: risk	M	story-4	3
13	Gateway, sesión de chat	Une tap, hooks y transcripción; WS de chat; enviar, detener, aprobar	M	story-11, story-12	4
14	App, pantalla de chat	`ChatItem` → `ThreadFeedEntry`, `ThreadFeed` de T3 o lista propia según la regla de salida; split: risk	M	story-9, story-13	5
15	App, escribir y aprobar	Editor, tarjeta de aprobación y preguntas, selector Chat/Terminal; split: file cap	M	story-14	6
16	Chat de Codex	Comprobación de `app-server`; base con rollout; palabra por palabra solo si es viable; split: risk	M	story-13	5
17	Push en el gateway (🚧 bloqueada hasta tener cuentas)	Registro y borrado de dispositivos, APNs y FCM, avisos de `blocked` y `done`, secretos; split: external dependency	M	story-4	3
18	Push en la app (🚧 bloqueada hasta tener cuentas)	Token del dispositivo, registro en cada gateway, capacidades de push, abrir el agente al tocar; split: external dependency, file cap	M	story-8, story-17	5

Definition of Done

Con el gateway instalado en `mylinux` y la app instalada en un teléfono Android y en el iPad del usuario, ambos dentro de su red de Tailscale:

1. En `mylinux`, correr `apps/herdr-gateway/scripts/install.sh`. Resultado: `systemctl --user is-active herdr-gateway herdr-tap` responde `active` dos veces, y el script imprime una URL `http://100.…:7788`.
2. Desde un dispositivo fuera de la cuenta de Tailscale del usuario (o con `curl` desde un nodo compartido), pedir `GET /v1/health`. Resultado: `403`.
3. En la app, "Agregar servidor" con esa URL. Resultado: aparece `mylinux` con los mismos espacios y pestañas que la app de escritorio Herdr, y las carpetas que el usuario creó en `space-folders`.
4. En el escritorio, pedirle a un Claude Code algo que requiera permiso. Resultado: en la app, ese agente sube a "Necesitan atención" en menos de 3 s.
5. Abrir el agente. Resultado: abre en Chat con el historial reciente; al pedirle algo desde el teléfono, el mensaje aparece en la terminal del escritorio y la respuesta llega palabra por palabra a la app.
6. Cuando Claude pida permiso, tocar "Permitir" en la app. Resultado: la terminal del escritorio continúa sin intervención.
7. Cambiar a Terminal. Resultado: la terminal se ve ajustada al ancho del teléfono, se puede escribir `ls` y ver la salida, y `^C` interrumpe.
8. En un espacio, "+" → Shell. Resultado: aparece una pestaña nueva en la app y en el escritorio.
9. Abrir Archivos, descargar un archivo y subir una imagen con el mismo nombre de un archivo existente. Resultado: la descarga abre la hoja de compartir; la subida queda como `nombre (1).ext` y el original no cambia.
10. Abrir un agente de Codex. Resultado: abre en Chat con sus mensajes (al menos por bloques).
11. Apagar el gateway de `mylinux` (`systemctl --user stop herdr-gateway`). Resultado: la app marca `mylinux` como "Sin conexión" y los agentes de Claude Code siguen funcionando en el escritorio. Al arrancarlo, la app se reconecta sola.
12. En iPad: barra lateral con la lista, contenido con el agente e inspector de archivos visibles a la vez.
13. Correr el workflow `herdr-upstream-sync` a mano. Resultado: se abre un PR con los cambios de T3, y `herdr-guard` pasa.

(Las notificaciones push quedan fuera de esta validación hasta desbloquear las historias 17 y 18.)

Testing Strategy

- [ ] Unit tests: tramas de `hc` (partidas, combinadas), `hello` y cambio de modo, créditos; nombres de espacio y carpetas; auth por `whois`; límites de rutas de archivos y nombres libres; parser SSE del tap; mapa sesión ↔ terminal; traducción de transcripción y rollout a `ChatItem`; `ChatItem` → `ThreadFeedEntry`; script de lista permitida.
- [ ] Integration tests: gateway contra un `HcConnection` falso que reproduce eventos grabados; tap contra un servidor SSE local que imita a Anthropic; receptor de hooks con secuencias grabadas de una sesión real; prueba de contrato opcional contra `hc schema` real (`HERDR_LIVE_HC=1`).
- [ ] Manual testing: comprobaciones con Claude real de las historias 11 y 12; build de Android en `mylinux` y de iOS en la Mac.
  (Feature-level end-to-end validation runs in `/test-feature` against the Definition of Done above — it is not a task in this plan.)

Risks & Considerations

- **Churn de T3:** las pantallas que reutilizamos (`ThreadFeed` , layout, terminal) cambian a diario. El chequeo de lista permitida evita conflictos en archivos de T3, pero un cambio de props en `ThreadFeed` o `TerminalSurface` puede romper la compilación de nuestro código en un PR semanal. Eso es lo esperado: se arregla en el PR.
- **El proxy es crítico:** si `herdr-tap` cae a media sesión, esa sesión de Claude falla hasta reiniciarla. Por eso es un proceso aparte, mínimo y sin estado. Usar `ANTHROPIC_BASE_URL` desactiva Remote Control de Anthropic en esas sesiones.
- **Alcance de la función `claude` :** `~/.zshenv` solo define la función `claude` (decisión 6); no exporta nada, así que ningún otro programa cambia. "Todo Claude Code" incluye los `claude` lanzados por zsh desde SSH no interactivo, como los pipelines de n8n y Taiga en `gx10` : si el tap no está vivo, no las afecta; si lo está, pasan por él. Un `claude` lanzado sin zsh (por ejemplo, desde otro programa con `exec`) va directo a Anthropic y no tiene palabra por palabra.
- **Diálogos de permiso duplicados:** no está comprobado cómo conviven el diálogo de la terminal y la espera del hook. La historia 12 lo comprueba antes de que la 13 dependa de ello; si responder en la terminal no termina la espera, se aplica el respaldo de 30 s (decisión 17).
- **`ThreadFeed` sin servidor T3:** no está comprobado que se pueda montar con datos propios. La historia 14 tiene una regla de salida.
- **Transcripción y rollout internos:** su formato puede cambiar con una actualización de Claude Code o Codex. Solo se usan para historial, con un lector tolerante.
- **Documento `herdr-app` :** es interno de Herdr y sin contrato público. Si cambia, el snapshot se rompe en un solo lugar (`snapshot.ts`).
- **Codex y `--no-daemon` :** el palabra por palabra de Codex puede resultar inviable; la base por bloques queda garantizada.
- **Cuentas:** sin Apple Developer de pago, la app en iPhone/iPad caduca a los ~7 días y no hay push; las historias 17 y 18 quedan bloqueadas.
- **Builds iOS:** solo en la Mac del usuario. Las historias de app se verifican en Android en `mylinux` y con pruebas unitarias; iOS se valida a mano.
- **Workflows de T3 en el fork:** se ejecutarían al activar Actions. Hay que deshabilitarlos con `gh workflow disable` (historia 1 lo documenta).
- **Repo:** el fork `kuamatzin/herdr-core-mobile` hoy viene de `benkraus` ; el usuario debe borrarlo y hacer fork de `pingdotgg/t3code` con ese nombre antes de subir la historia 1.
- **Rollback:** desinstalar con `apps/herdr-gateway/scripts/install.sh --uninstall` : detiene los servicios, quita la línea de `.zshenv` y las entradas marcadas de `~/.claude/settings.json` .

Estimated Total Effort

18 historias: 2 S y 16 M. Son más de 6 porque el plan cubre cuatro sistemas (gateway, app, proxy con hooks, push) y aísla en historias propias los cuatro riesgos sin comprobar (proxy, hooks, montar `ThreadFeed` , Codex), para que un fallo en uno no bloquee la base. La base usable (historias 1–10) son 10 historias; el chat, 6; push, 2.